CAOS — Composable Cognitive Architecture

Vision, RulePacks, specialized tools, contextual behavior and research directions

AOSA / CAOS Architecture Notes

28 October 2026

Contents

1 Executive thesis **1**

2 1. Architectural foundations to preserve **2**

3 2. Tools as the main reusable unit **3**

4 3. RulePacks: what they are, and what they are not **3**

5 4. Current Writer tool: concrete implementation reference **4**

6 5. From individual consumers to reusable RulePack tools **4**

7 6. Context, channels and automatic applicability **5**

8 7. Image quality and market-specific requirements **6**

9 8. Personality as compositional cognition **6**

10 9. Recursive cognitive organization: the research horizon **7**

11 10. Practical engineering criteria **7**

12 11. Development continuity and proposed sequencing **7**

13 12. Core statement of intent **8**

Document status: Vision and architecture research note. This document captures the project owner's conceptual direction discussed on 28 October 2026. It deliberately separates observed implementation patterns, previously established architectural decisions and proposed future capabilities. It is **not** an implementation specification, policy certification or claim of experimental validation.

Suggested repository destination: `docs/architecture/exoteric/research/COMPOSABLE_COGNITIV` with links from the exoteric architecture index and the esoteric research index as appropriate. Repository integration and ADR approval remain separate actions.

1 Executive thesis

CAOS (Cognitive Agentic Operative System) is not merely a system for executing autonomous agents. It is an operating system for composing governed, specialized capa-

bilities into context-sensitive agent behavior, enterprise processes and experimental cognitive systems.

AOSA (Agent Oriented System Architecture) provides the architectural and research foundation. **CAOS** is its operative-system implementation. **IAOS** remains the earlier system and should not be conflated with CAOS.

The preferred engineering direction is to implement as much reusable behavior as practical in **small, registered tools**, while retaining declarative descriptions of agents, personalities, RulePacks and workflows. An agent is a persistent identity that composes authorized capabilities; it is not necessarily an enormous program or a monolithic LLM prompt.

The guiding formula is conceptual, not an executable contract:

> **Situated behavior = f(identity, cognitive capabilities, context, personality, RulePacks, relationships, evidence), subject to independent trusted authorization and enforcement.**

This direction aims to improve maintainability, reuse, observability, testing and governance, while enabling controlled research into compositional personality and social cognition. It does **not** imply that every action becomes an agent, network call, RabbitMQ event or LLM request.

2 1. Architectural foundations to preserve

The established division of responsibilities remains:

Component	Responsibility
ToolSpec and registries	Define and resolve trusted capability and resource contracts.
UTU	Own identity, authorization, tenant and scope boundaries, trusted delegation and revocation.
NCSC/CPBX	Connect and invoke registered capabilities over explicitly governed synaptic lines.
Enlil / OPS V2	Supervise durable operation lifecycle, cancellation, recovery and finalization.
WFT V2	Orchestrate workflow dependencies, step ordering and bindings; not security authorization.
Agent Soul	Hold persistent identity and relevant personal configuration, dispositions and references.
Cognitive tools	Perform specialized work, potentially deterministic or model-assisted.
RulePacks	Carry declarative boundaries, instructions, criteria and expectations to be consumed by tools.

Operating maxim: *Registry resolves. UTU authorizes. NCSC connects. Enlil supervises. WFT orchestrates.*

No RulePack, personality preference, workflow JSON, prompt or event grants system authority. A trusted line controls *who can invoke whom*; workflow dependency controls *when a step should run*. Privileged mechanisms should be deterministic where possible.

3 2. Tools as the main reusable unit

A tool is not limited to a third-party API. It can perform cognitive and operational work: collecting evidence, retrieving memory, evaluating an instruction, resolving a RulePack, composing a prompt, analyzing a claim or assessing image quality.

Examples of **proposed** synaptic capabilities:

- `rulepack.resolve`: retrieve approved registered RulePacks and relevant metadata.
- `rulepack.evaluate`: evaluate a concrete artifact, situation or proposal against applicable criteria.
- `prompt.create`: compose a prompt/context from approved task, personality, rules and evidence.
- `personality.infuse`: supply relevant personality dispositions without redefining factual evidence.
- `context.appraise`: identify situational relevance and competing non-security priorities.
- `memory.retrieve`: retrieve context-appropriate, authorized history.
- `image.quality.evaluate`: inspect an image against measurable quality requirements and declared target-market standards.

These are **illustrative proposals**, not confirmed registered handlers. The smallest useful implementation may expose only `rulepack.resolve` and `prompt.create` initially; standalone `evaluate` or `compose` tools should be justified by reuse and clear semantics.

A cognitive capability can be a pure function, validator, repository-backed service, small classifier, LLM adapter or an independent agent. **Capability boundaries do not require LLM boundaries.** Low-latency local invocation should be preferred when durable workflow infrastructure would create needless overhead.

4 3. RulePacks: what they are, and what they are not

A **RulePack is a declarative JSON-shaped resource** described by an existing RulePack Spec and Registry. It can provide any reusable set of instructions, boundaries, quality criteria or behavioral expectations relevant to an activity. The concept should not be restricted to compliance or agent personality.

Examples spanning the same abstraction:

Context	RulePack describes
Green-claim text	What constitutes supported or unsupported sustainability statements, and how to write or audit claims.
Writer	Style, voice, task and response constraints, such as avoiding questions in the final answer.
Weather interpretation	Preserve observed weather facts; distinguish forecasts and uncertainty.
Image production	Minimum image quality, composition, technical delivery specifications and market-specific visual expectations.
Physical tool use	Proper handling and injury precautions (instructions alone cannot guarantee physical safety).
Corporate workflow	Applicable internal review criteria, policy requirements and standards.

Regulated market	Relevant documented requirements, versions, jurisdictions and review obligations.

ToolSpec describes what a capability does. RulePack describes how it is expected to be used or what its result should satisfy. A consuming tool decides how its relevant RulePack content is interpreted: as LLM instructions, deterministic constraints, evaluation criteria or an audit rubric. This interpretation must be explicit and testable.

RulePacks must not become shadow security policies. UTU continues to authorize access; any mandatory safety or business gate needs actual trusted enforcement. A prompt saying "must comply" does not establish compliance.

5 4. Current Writer tool: concrete implementation reference

The published CAOS `alpha` implementation already demonstrates this approach:

1. `WriterTool.provide_context()` parses the invocation and determines applicable RulePack IDs.
2. `resolve_rule_context()` reads either an explicit RulePack override or the agent's configured `rule_pack_ids`, resolves packs through the RulePack Registry and collects rules.
3. `build_writer_system_message()` turns the selected rule descriptions and Writer-specific pack configuration into generation instructions.
4. `assemble_context_bundle()` combines the system instructions, writing task, available history and model hints; the Writer appends RulePack evidence for traceability.
5. The LLM performs the writing task; the Writer formats the response.

Relevant source files (paths relative to the repository):

- `src/domains/agentverse/tools/writer_tool/__init__.py`
- `src/domains/agentverse/tools/_helpers/rule_context.py`
- `src/domains/agentverse/tools/writer_tool/sys_message.py`
- `src/domains/agentverse/tools/writer_tool/messages.py`
- `src/domains/agentverse/agents/roles/rules/rulepack_spec.py`
- `src/domains/agentverse/registries/rulepack_registry.py`

Present limitations to retain in the design record: Explicit override IDs replace the agent defaults in this resolver; unresolved pack IDs are reported but do not themselves stop execution; writing constraints are expressed in prompts rather than independently guaranteed. These are factual observations about this Writer path, not a complete review of all security checks elsewhere. Mandatory inherited RulePacks would need stricter resolution and enforcement behavior.

6 5. From individual consumers to reusable RulePack tools

The natural, incremental evolution is to extract shared behavior without discarding the Writer pattern:

```
Trusted execution context
  (principal, tenant, agent, tool, environment, market, operation)
       |
       v
RulePack applicability / trusted selection
       |
       v
```

```
rulepack.resolve  ----> approved, versioned rule context
       |
       +----> prompt.create ----> model-assisted work
       |
       +----> rulepack.evaluate ----> result / findings / evidence
       |
       +----> deterministic execution or validation gates
```

Resolution is not evaluation. `rulepack.resolve` selects and retrieves definitions. `rulepack.evaluate` assesses a specific artifact or situation. `prompt.create` constructs the model's working context from approved inputs. Different consumers should not duplicate registry lookup or basic normalization, but each may retain domain-specific application logic.

Proposed structured resolution output should carry pack ID, version, source, whether selection is mandatory, applicable rule IDs, provenance and unresolved references. Treat this as a proposed contract, not an already adopted schema.

7 6. Context, channels and automatic applicability

The examples `channel.weather.rulepack` and `channel: jail` / `rulepack: jail-rulepack-inmate` illustrated **context-dependent applicability**, not a requirement to invent a new channel runtime. The same principle applies to practical enterprise environments.

A channel or environment label can describe the situation in which a capability is operating. Applicability can depend on jurisdiction, target market, tenant, organization, department, role, tool, workflow step, operation and output artifact. Two participants may be subject to different role-specific rules in the same context.

Expected target behavior: When an agent or workflow executes under a governed context, applicable mandatory rules are resolved automatically from trusted metadata. A user cannot silently remove mandatory corporate, jurisdictional or operational requirements with a discretionary RulePack override.

A conceptual example (not the current RulePack Spec):

```
{
  "rulepack": "market.se.image_delivery",
  "applies_when": {
    "market": ["SE"],
    "capability": ["image.create", "image.quality.evaluate"]
  },
  "mandatory": true
}
```

Separate the following decisions:

- **Applicability:** Which RulePacks are relevant to this context?
- **Authorization:** May the requesting principal read or invoke the resource/capability?
- **Interpretation:** Which tool applies the instructions and how?
- **Enforcement:** Which criteria are deterministically checked, semantically assessed or reviewed by a person?
- **Evidence:** Which pack/version, rule and artifact motivated the final decision?

For mandatory packs, failure to resolve or check a necessary requirement should lead to rejection, explicit review or another defined fail-closed path rather than silent omission.

8 7. Image quality and market-specific requirements

The image example demonstrates that RulePacks extend beyond text and agents. A creative workflow may need to produce an image that satisfies both artistic and operational criteria.

A single image pipeline might consult multiple registered resources:

```
image.create
  + brand.visual_identity
  + market.target_requirements
  + channel.image.quality
        |
        v
image.quality.evaluate
        |
        +-- measurable technical checks (dimensions, format, etc.)
        +-- aesthetic/semantic assessment where appropriate
        +-- provenance and confidence
        |
        v
approve / revise / route for human review
```

Some criteria are easy to verify mechanically (dimensions, format, aspect ratio). Others, such as whether visual communication is culturally appropriate or legally compliant in a target market, need reliable domain evidence, evaluation design and possibly expert review. An LLM evaluation score alone is insufficient to certify compliance.

9 8. Personality as compositional cognition

A research hypothesis is that coherent situated personality might be modeled as interactions among specialized cognitive functions: perception, memory, reasoning, empathy, value evaluation, creativity and expression. This is a computational analogy, **not** a claim that human personality literally consists of software-like agents.

An Agent Soul supplies persistent identity and dispositions. Context determines which cognitive functions become relevant. Relationships and memory affect appraisal; RulePacks guide expected conduct. Governance independently bounds actual operations.

The previously discussed BIG-inspired persona example makes the distinction vivid: the same identity might express itself differently on stage, with a friend or with a family member. Context changes which aspects of behavior become salient, rather than replacing identity. Similarly, a weather and clothing task may favor style in planning and expression while keeping weather evidence objective.

Identity belongs to the agent. Relationships belong to interaction edges. Those edges may be directional and history-dependent. Social trust/familiarity is not UTU authorization.

Specialized cognitive units may interact like participants in a small internal society. Some capabilities reinforce one another; some inhibit or prioritize alternatives. Behavior need not be a simple numeric average or compromise. Complex cognition should not require an independent LLM call for each component.

10 9. Recursive cognitive organization: the research horizon

The broader AOSA research program asks whether similar organizing principles recur across scales:

1. **Cognitive units** compose functions or faculties.
2. **Faculties** contribute to an individual agent's coherent behavior.
3. **Agents** form relationships and collaborative teams.
4. **Teams** develop shared norms, coordination and institutional constraints.
5. **Organizations and simulated societies** exhibit higher-level patterns of stability, conflict and adaptation.

Candidate research lines include **Composable Personality**, **Context-Adaptive EQ**, **Relational Cognition**, **Emergent Civilizations**, **Recursive Cognitive Organization** and **Cognitive Homeostasis**. These are proposals rather than implemented results.

A reasonable controlled experiment compares a monolithic personality prompt, fixed modular composition and context-adaptive modular composition under identical tasks, model access, tool sets and memory snapshots. Measure factual accuracy, identity consistency, contextual appropriateness, rule adherence, cost, latency, reproducibility and failure modes. More modules are not automatically better.

The philosophical/Hermetic inspiration belongs to the project's *esoteric research perspective*. Its metaphysical interpretation should not be presented as empirical proof about consciousness or reality. The same software architecture must stand on its *exoteric* enterprise engineering merits.

11 10. Practical engineering criteria

The following guardrails keep this vision implementable:

- **Prefer existing primitives first.** Before creating a new engine or supervisor, inspect specs, registries, tools, NCSC and UTU.
- **Small, typed, versioned contracts.** Inputs, outputs, provenance and error behavior must be explicit.
- **Keep data declarative.** RulePacks and personalities are configuration/resources, not hidden executable authority.
- **Mandatory versus optional.** Trusted required constraints cannot be removed by client flags or prompts.
- **Deterministic by default.** Use models for semantic tasks, not registry discovery, privilege granting or hard enforcement.
- **No unnecessary distributed hops.** Not every cognitive step requires durable operation supervision, an event or an LLM.
- **Observability and replayability.** Capture versions, selected rules, tool invocations, evidence and outcomes without indiscriminate sensitive-data logging.
- **Separation of concerns.** Enlil supervises operations; WFT sequences workflows; UTU alone owns security authority.
- **Evidence before complexity.** Introduce new cognitive tools only when testing shows concrete advantages over simple in-process behavior.

12 11. Development continuity and proposed sequencing

This document is a design and research record, **not a request to change the immediate development milestone**.

At the time of this note, development continues on `feature/ops-v2`. The reported documentation reconciliation preserved code, security policies and ADR status, verified Markdown navigation and ran focused tests (143 passed, 11 warnings). Remaining reported documentation follow-ups include historically inconsistent NCSC README language and pending research sources; those must be reconciled against the actual working tree rather than assumed completed.

Immediate implementation priorities remain:

1. Finish documentation consistency and review changes before committing.
2. Resume **OPS2-01**: operation identity, attempt identity, state transitions and Enlil lifecycle contracts.
3. Develop WFT V2 on stable OPS V2 foundations.
4. Prove **Genesis: JOSHUA → Enki → Adam** under governed orchestration and activation.
5. Only then consider a focused, isolated demonstration of shared RulePack resolution / `prompt.create` with deterministic applicability and observable evidence.

The research concepts here should inform designs and documentation, without prematurely becoming requirements for Genesis or displacing its security and execution milestones.

13 12. Core statement of intent

> **CAOS should not hardcode intelligence into one agent or one LLM prompt. It should provide the governed conditions in which reusable specialized capabilities, declarative resources, contextual boundaries and persistent identities can be composed, observed, tested and improved.**
>
> **AOSA studies which such compositions produce useful, coherent behavior. CAOS implements the secure operating system in which those compositions can run.**

Provenance and scope: Synthesized from the project owner's October 2026 architectural discussion, the supplied *CAOS Architecture Decisions and Conversation Continuity* PDF (27 October 2026) and inspected `alpha` source for the Writer / RulePack pattern. The *Brain Inspired Agentic OS* and *CAOS Hermeticum Volume I* PDFs are part of the supplied project corpus but are not claimed as line-by-line evidence for every statement above. This document intentionally labels proposed interfaces and hypotheses, and does not claim changes to the repository.